

Suvenir

Amaru sedang membeli souvenir di sebuah toko asing. Terdapat N **tipe** souvenir. Di toko tersebut, setiap tipe souvenir tersedia dalam jumlah yang tak terhingga.

Setiap tipe souvenir memiliki harga tetap. Harga dari souvenir tipe i ($0 \leq i < N$) adalah $P[i]$ koin, di mana $P[i]$ adalah sebuah bilangan bulat positif.

Amaru mengetahui bahwa tipe souvenir terurut menurun berdasarkan harga, dan harga souvenir berbeda-beda. Lebih tepatnya, $P[0] > P[1] > \dots > P[N-1] > 0$. Selain itu, Amaru juga mengetahui nilai dari $P[0]$. Sayangnya, Amaru tidak mengetahui harga-harga souvenir lainnya.

Untuk membeli souvenir, Amaru akan melakukan sejumlah transaksi dengan penjual.

Setiap transaksi terdiri dari langkah-langkah berikut:

1. Amaru memberikan sejumlah (positif) koin kepada penjual.
2. Penjual menumpuk koin di sebuah meja di ruang tertutup di mana Amaru tidak bisa melihatnya.
3. Penjual mempertimbangkan setiap tipe souvenir $0, 1, \dots, N-1$ sesuai urutan tersebut, satu per satu. Setiap tipe dipertimbangkan **tepat sekali** pada setiap transaksi.
 - Saat mempertimbangkan souvenir tipe i , jika terdapat setidaknya $P[i]$ koin pada tumpukan, maka
 - penjual mengambil $P[i]$ koin dari tumpukan, dan
 - penjual meletakkan satu souvenir tipe i di atas meja.
4. Penjual memberikan Amaru seluruh sisa koin di tumpukan dan seluruh souvenir yang berada di atas meja.

Perhatikan bahwa tidak terdapat koin maupun souvenir di atas meja sebelum sebuah transaksi dimulai.

Tugas Anda adalah menginstruksikan Amaru untuk melakukan sejumlah transaksi, sehingga:

- pada setiap transaksi, Amaru membeli **setidaknya satu** souvenir; dan
- secara keseluruhan, Amaru membeli **tepat i** buah souvenir tipe i , untuk setiap i sehingga $0 \leq i < N$. Perhatikan bahwa ini berarti Amaru tidak boleh membeli souvenir tipe 0.

Amaru tidak harus meminimalkan banyaknya transaksi, dan memiliki jumlah koin yang tak terbatas.

Detail Implementasi

Anda harus mengimplementasikan prosedur berikut.

```
void buy_souvenirs(int N, long long P0)
```

- N : banyaknya tipe souvenir.
- $P0$: nilai $P[0]$.
- Prosedur ini dipanggil tepat sekali untuk setiap kasus uji.

Prosedur ini dapat memanggil prosedur berikut untuk menginstruksikan Amaru untuk melakukan sebuah transaksi:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : banyaknya koin yang diberikan oleh Amaru kepada penjual.
- Prosedur ini mengembalikan sebuah *pair*. Elemen pertama dari *pair* tersebut merupakan sebuah *array* L yang mengandung tipe-tipe souvenir yang telah dibeli (secara terurut menaik). Elemen kedua dari *pair* tersebut merupakan sebuah bilangan bulat R yang menyatakan banyaknya koin yang dikembalikan ke Amaru setelah transaksi selesai.
- Dibutuhkan $P[0] > M \geq P[N - 1]$. Kondisi $P[0] > M$ menjamin bahwa Amaru tidak membeli souvenir tipe 0, dan $M \geq P[N - 1]$ menjamin bahwa Amaru membeli setidaknya satu souvenir. Jika kondisi tersebut tidak terpenuhi, maka solusi Anda akan mendapatkan *verdict* Output isn't correct: Invalid argument. Tidak seperti $P[0]$, perhatikan bahwa nilai dari $P[N - 1]$ tidak diberikan pada masukan.
- Prosedur ini dapat dipanggil paling banyak 5000 kali untuk setiap kasus uji.

Perilaku *grader* **tidak adaptif**. Artinya, nilai dari barisan harga P sudah ditentukan sebelum `buy_souvenirs` dipanggil.

Batasan

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ untuk setiap i sehingga $0 \leq i < N$.
- $P[i] > P[i + 1]$ untuk setiap i sehingga $0 \leq i < N - 1$.

Subsoal

Subsoal	Nilai	Batasan Tambahan
1	4	$N = 2$
2	3	$P[i] = N - i$ untuk setiap i sehingga $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ untuk setiap i sehingga $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ untuk setiap i sehingga $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ untuk setiap i sehingga $0 \leq i < N - 1$.
6	33	Tidak ada batasan tambahan.

Contoh

Perhatikan pemanggilan berikut.

```
buy_souvenirs(3, 4)
```

Terdapat $N = 3$ tipe souvenir dan $P[0] = 4$. Perhatikan bahwa terdapat tiga kemungkinan barisan harga P : $[4, 3, 2]$, $[4, 3, 1]$, dan $[4, 2, 1]$.

Asumsikan bahwa `buy_souvenirs` memanggil `transaction(2)`. Misalkan pemanggilan tersebut mengembalikan nilai $([2], 1)$, yang berarti Amaru membeli satu buah souvenir tipe 2 dan penjual mengembalikan 1 koin. Perhatikan bahwa hal ini memungkinkan kita untuk menyimpulkan bahwa $P = [4, 3, 1]$, karena:

- Untuk $P = [4, 3, 2]$, `transaction(2)` seharusnya mengembalikan $([2], 0)$.
- Untuk $P = [4, 2, 1]$, `transaction(2)` seharusnya mengembalikan $([1], 0)$.

Setelah itu, `buy_souvenirs` dapat memanggil `transaction(3)` yang mengembalikan $([1], 0)$. Artinya, Amaru membeli satu buah souvenir tipe 1 dan penjual mengembalikan 0 koin. Sejauh ini, Amaru telah membeli satu buah souvenir tipe 1 dan satu buah souvenir tipe 2.

Akhirnya, `buy_souvenirs` dapat memanggil `transaction(1)` yang mengembalikan $([2], 0)$. Artinya, Amaru membeli satu buah souvenir tipe 2. Perhatikan bahwa kita juga dapat memanggil `transaction(2)` di sini. Pada titik ini, Amaru sudah memiliki satu buah souvenir tipe 1 dan dua buah souvenir tipe 2, sesuai yang dikehendaki.

Contoh Grader

Format masukan:

N

$P[0] \ P[1] \ \dots \ P[N-1]$

Format keluaran:

$Q[0] \ Q[1] \ \dots \ Q[N-1]$

Di sini, $Q[i]$ menyatakan banyaknya souvenir tipe i yang dibeli secara keseluruhan untuk setiap i sehingga $0 \leq i < N$.